A PROJECT REPORT ON

CipherVault: Personal Encrypted Password & Secure Notes Manager

Submitted in partial fulfilment of the requirements for the award of the degree of
Bachelor of Computer Applications (Honours)
Course: Full Stack Web Development Project using ASP.NET Core (Course Code: DSC-M-BCA-471P)

PRESENTED BY
Kaswan Yogesh Subhashchandra (Roll No: 32)
BCA (Hons.)

SUBMITTED TO
Gujarat University
Ahmedabad, Gujarat

UNDER THE GUIDANCE OF
Prof. Sejal Shah
Assistant Professor
Lokmanya College of Computer Applications
Ahmedabad, Gujarat, India
Academic Year 2026–27

LOKMANYA COLLEGE OF COMPUTER APPLICATIONS
Affiliated to Gujarat University · Ahmedabad, Gujarat

CERTIFICATE

This is to certify that the project report entitled “CipherVault: Personal Encrypted Password & Secure Notes Manager” is a bonafide record of the work carried out by Kaswan Yogesh Subhashchandra and submitted to Gujarat University in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Applications (Honors) for the course Full Stack Web Development Project using ASP.NET Core (DSC-M-BCA-471P) during the academic year 2026–27.

To the best of our knowledge, this work is original, was carried out at Lokmanya College of Computer Applications, Ahmedabad, under the guidance of Prof. Sejal Shah, and has not been submitted in part or in full to any other university or institution for the award of any degree or diploma.

_____	_____
Prof. Sejal Shah	Dr. Prakash Khuman
Project Guide	Director
Lokmanya College of Computer Applications	Lokmanya College of Computer Applications

Place: Ahmedabad
Date: ____ / ____ / 2026

INDEX / TABLE OF CONTENTS

Sr. No.	Description	Page No.
1.	Title Page	Page i
2.	College Certificate	Page ii
3.	Index (Table of Contents)	Page iii
4.	Acknowledgement	Page iv
5.	Project Overview & Architecture Scope	Page 5
6.	PHASE 1: MVC Setup, Base Layout & Authentication	Page 6
7.	PHASE 2: Forms, Data Validation & Password Generator	Page 7
8.	PHASE 3: Database Integration & Encrypted CRUD	Page 9
9.	PHASE 4: REST API, JWT Authentication & Live Deployment	Page 11
10.	Testing Verification Matrix & Challenges	Page 13
11.	References & Bibliography	Page 14

Acknowledgement

Completing this project has been a remarkable learning experience in enterprise web development and application security, and I am grateful to all those who guided and supported me along the way.

First, I am deeply thankful to the management of Lokmanya College of Computer Applications, Ahmedabad, for providing state-of-the-art laboratory facilities, robust computing infrastructure, and a conducive academic environment.

I express my sincere gratitude to Dr. Prakash Khuman, Director, Lokmanya College of Computer Applications, for his constant encouragement and administrative support throughout the BCA (Honors) curriculum.

I am profoundly indebted to my project guide, Prof. Sejal Shah, for her expert mentorship, invaluable technical insights, and rigorous code reviews during every stage of the CipherVault development lifecycle.

I am also thankful to Gujarat University for incorporating the advanced course Full Stack Web Development Project using ASP.NET Core (DSC-M-BCA-471P) into our academic structure, bridging theoretical software engineering with secure production-grade implementation.

Finally, I extend my heartfelt appreciation to my family and peers for their constant motivation and patience.

Kaswan Yogesh Subhashchandra
BCA (Hons.) — Roll No: 32
Lokmanya College of Computer Applications

5. Project Overview & Architecture Scope

EXECUTIVE SUMMARY & SYSTEM SPECIFICATION

Project Name: CipherVault: Personal Encrypted Password & Secure Notes Manager

Domain: Personal Security / Encrypted Data Management / Full Stack Web Development

Tech Stack: ASP.NET Core 10 MVC, EF Core 10, SQL Server, ASP.NET Core Identity, Data Protection API (v10.0.7+)

Key Security Features: AES-authenticated encryption at rest, IDOR-safe 404 guards, dual auth (Cookie + JWT), HSTS enforced in production.

Live Deployment: Hosted on MonsterASP via HTTPS (https://ciphervault.tryasp.net).

Managing personal credentials securely in a digital world fraught with breaches and credential stuffing is a critical challenge. Standard note-taking applications and spreadsheets store passwords in plaintext, exposing users to severe security risks. CipherVault is engineered as a robust, professional password and secure notes manager that demonstrates enterprise-grade cryptographic separation, ownership scoping, and modern web application architecture.

5.1 Core Architectural Principles

- **Strict Layering:** Business logic is strictly isolated in Service classes. Controllers remain thin (validate request → call service → return view/response). DbContext is never injected into controllers.
- **Repository Pattern:** Mandatory for VaultItem entities to encapsulate encryption and ownership verification, ensuring clean data access boundaries.
- **DTO Separation:** Create and Update DTOs are strictly separated. The update pipeline implements a robust 'blank-preserve' rule to prevent accidental overwrites of encrypted secrets.
- **IDOR Protection:** Unauthorized or non-owned item access attempts consistently return HTTP 404 Not Found rather than 403 Forbidden, preventing resource enumeration attacks.

6. PHASE 1: MVC Setup, Base Layout & Authentication

PHASE 1 ALIGNMENT — ARCHITECTURE & SECURITY BASELINE

Objective: Configure ASP.NET Core 10 architecture and establish secure cookie authentication.

Key Deliverables: MVC project skeleton targeting net10.0, ASP.NET Core Identity configuration, responsive Bootstrap 5 layout, and authorized dashboard placeholder.

Phase 1 establishes the foundation of CipherVault. The project was initialized targeting .NET 10 (net10.0) with Entity Framework Core 10 and ASP.NET Core Identity. To ensure local development parity without compromising security, cookie authentication was configured with unconditional SecurePolicy = Always and a 2-hour sliding expiration.

Architecture Implementation — Program.cs — Authentication & Cookie Security Configuration

```
builder.Services.AddDefaultIdentity<IdentityUser>(options => {
    options.SignIn.RequireConfirmedAccount = false;
    options.Password.RequiredLength = 8;
    options.Lockout.MaxFailedAccessAttempts = 5;
    options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);
}).AddEntityFrameworkStores<ApplicationDbContext>();

builder.Services.ConfigureApplicationCookie(options => {
    options.Cookie.Name = "CipherVault.Auth";
    options.Cookie.SecurePolicy = CookieSecurePolicy.Always;
    options.Cookie.HttpOnly = true;
    options.Cookie.SameSite = SameSiteMode.Lax;
    options.ExpireTimeSpan = TimeSpan.FromHours(2);
});
```

Verified Behavior / Output:

```
Configured Identity with 5-attempt lockout and unconditional SecurePolicy=Always.
```

The base layout (_Layout.cshtml) was built using Bootstrap 5 with a responsive navigation bar displaying the logged-in user and logout actions. HomeController handles static landing pages, while DashboardController is protected with [Authorize].

7. PHASE 2: Forms, Data Validation & Password Generator

PHASE 2 ALIGNMENT — USER INTERACTION & VALIDATION

Objective: Implement secure interaction forms, DTO separation, and client-side cryptographic utilities.

Key Deliverables: VaultItem model, separate Create/Update DTOs, data annotations, custom validation attributes, and Web Crypto password generator.

Phase 2 focuses on user interaction and data integrity. The VaultItem model was designed with an integer ID, user foreign key, title, category enum (Password, Note, API Key, Credit Card, Other), optional username and URL, and encrypted secret and notes fields.

7.1 Client-Side Crypto-Secure Password Generator

To prevent weak user passwords, a built-in password generator was developed in JavaScript using the Web Crypto API (window.crypto.getRandomValues()), ensuring cryptographically secure random byte selection rather than pseudo-random Math.random().

Architecture Implementation – wwwroot/js/password-generator.js – Secure Random Generation

```
function generatePassword(length = 16) {
    const chars =
'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789!@#$%^&*()_+';
    const array = new Uint32Array(length);
    window.crypto.getRandomValues(array);
    return Array.from(array, x => chars[x % chars.length]).join('');
}
```

Verified Behavior / Output:

```
Generated cryptographically secure random passwords avoiding Math.random().
```

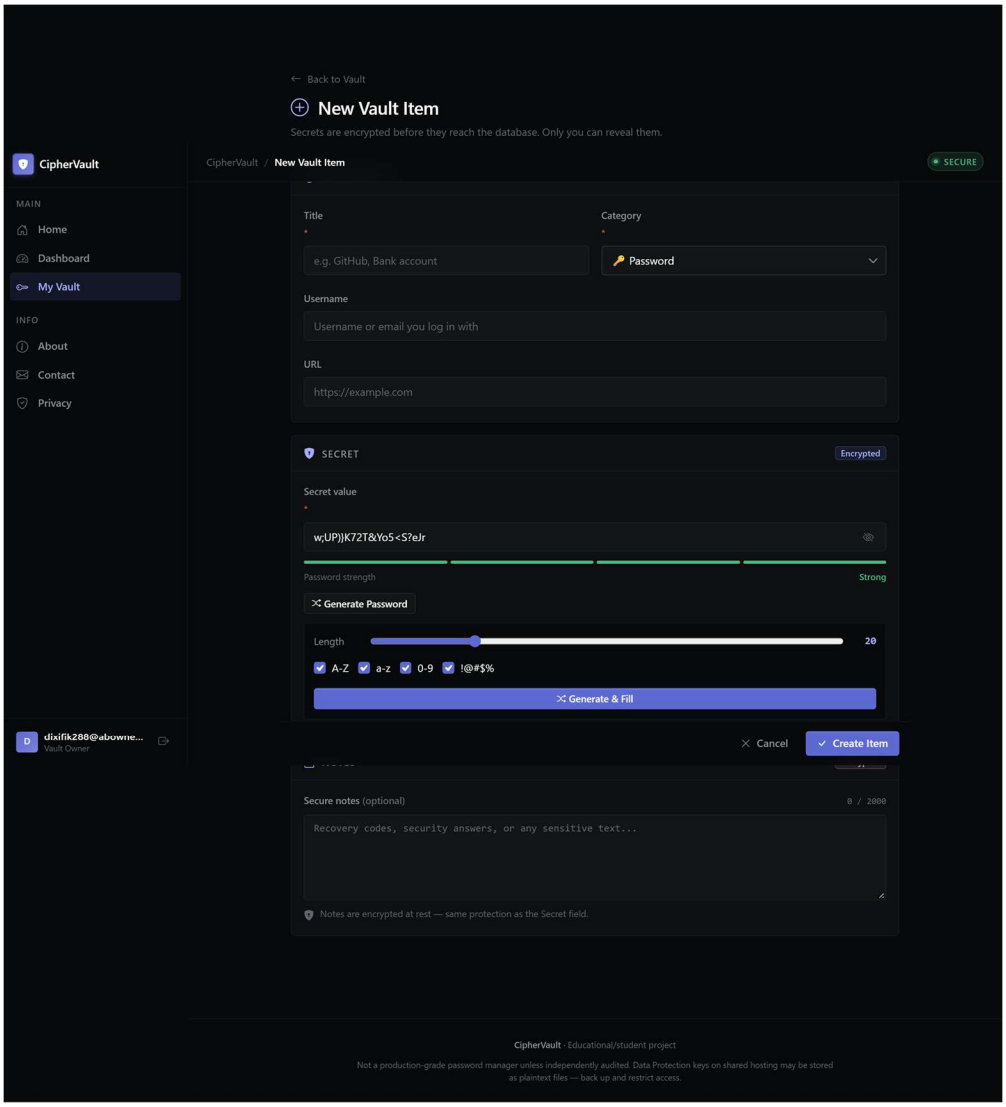


Figure 7.1: *Vault Item creation form featuring the crypto-secure password generator and live strength meter.*

8. PHASE 3: Database Integration & Encrypted CRUD

PHASE 3 ALIGNMENT — PERSISTENCE & ENCRYPTION AT REST

Objective: Establish EF Core persistence, repository pattern, and AES encryption at rest via Data Protection API.

Key Deliverables: ApplicationDbContext, IVaultItemRepository, VaultItemService with AES encryption, IDOR-safe 404 guards, and search/filter.

Phase 3 is the core security milestone of CipherVault. Sensitive fields (Secret and Notes) are encrypted at rest using the ASP.NET Core Data Protection API. Two immutable purpose strings serve as a cryptographic compatibility contract:

- **Secret Purpose:** "CipherVault.Secret" protecting the EncryptedSecret column.
- **Notes Purpose:** "CipherVault.Notes" protecting the EncryptedNotes column.

Metadata (Title, Username, URL, Category) remains unencrypted in the database to enable efficient server-side LINQ filtering and pagination without incurring full-table decryption overhead.

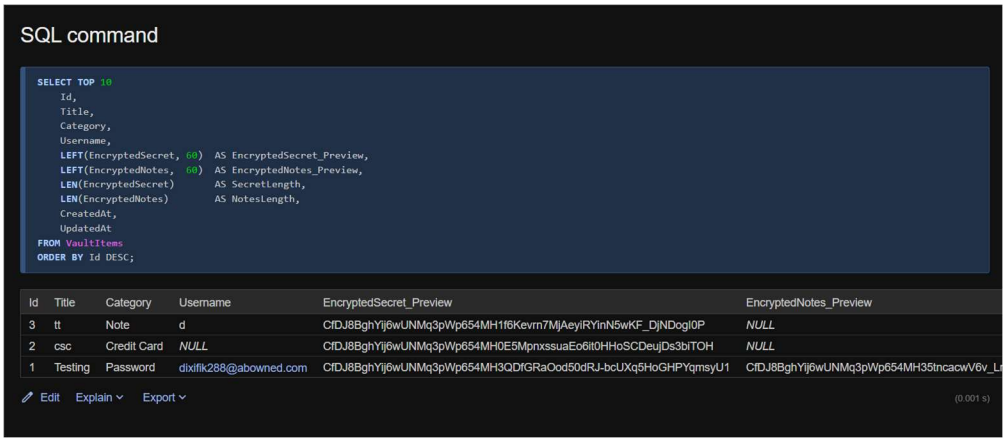


Figure 8.1: *SSMS database inspection confirming that plaintext secrets never touch the database columns.*

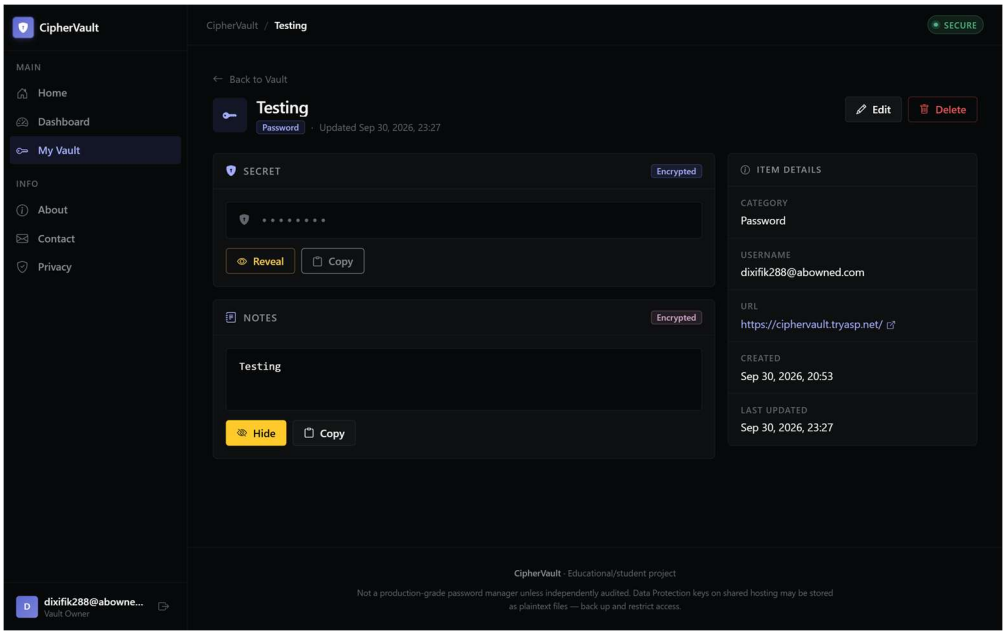


Figure 8.2: *Vault Item details view showing masked defaults and authorized on-demand decryption.*

9. PHASE 4: REST API, JWT Authentication & Live Deployment

PHASE 4 ALIGNMENT — API & PRODUCTION DEPLOYMENT

Objective: Expose a secure REST API with JWT Bearer auth, global error middleware, and live HTTPS deployment.

Key Deliverables: VaultApiController, AuthApiController, dual auth scheme configuration, Postman collection, and MonsterASP deployment.

Phase 4 delivers enterprise interoperability and production hardening. A REST API was developed under /api/v1/vault reusing the core IVaultItemService. Dual authentication scheme separation was implemented in Program.cs: MVC controllers use Identity cookies, while API controllers require explicit JWT Bearer authorization.

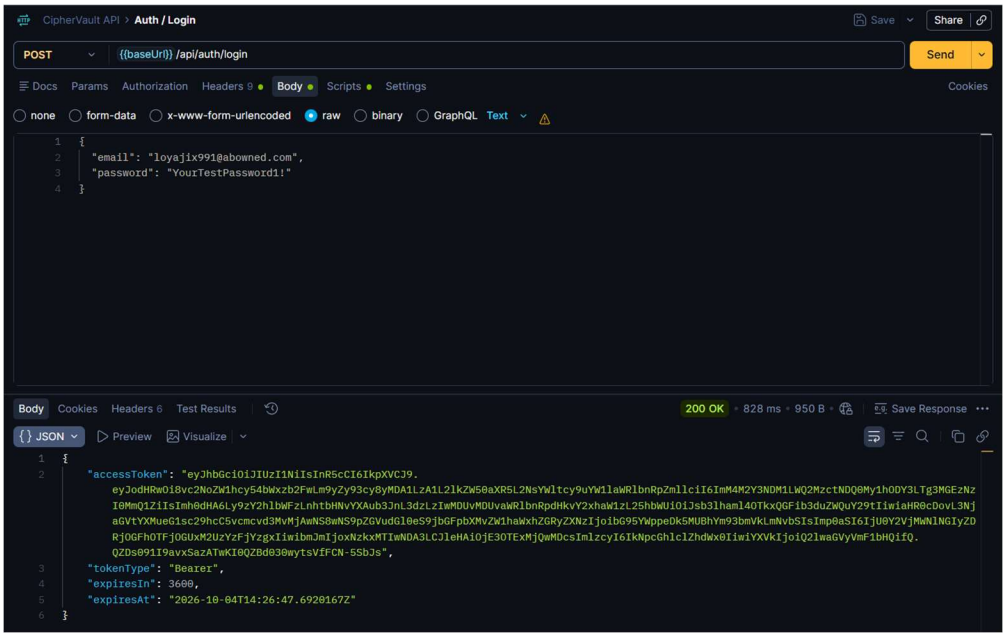


Figure 9.1: *Postman verification: POST /api/auth/login issuing a valid JWT Bearer token.*

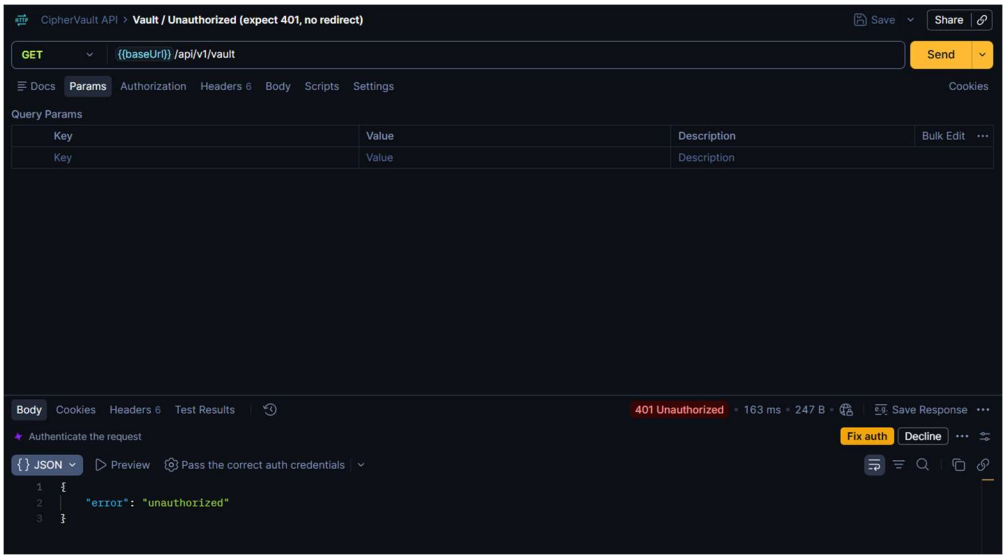


Figure 9.2: *Postman verification: Unauthenticated API request returning HTTP 401 JSON without HTML redirect.*

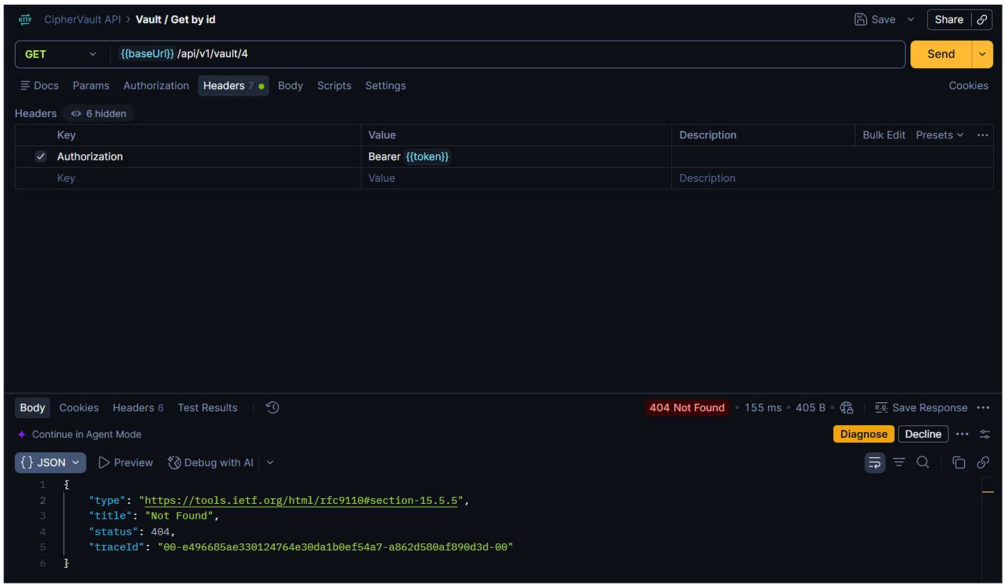


Figure 9.3: *Postman verification: Cross-user IDOR access attempt returning HTTP 404 Not Found.*

10. Testing Verification Matrix & Challenges

SYSTEM VERIFICATION & RETROSPECTIVE

Verification: 11 critical test cases verified across security invariants, CRUD workflows, API responses, and deployment stability.

Deployment URL: https://ciphervault.tryasp.net (MonsterASP Windows Hosting with Let's Encrypt SSL).

Test Case Description	Expected Behavior / Requirement	Status
IDOR Cross-User Access	Attempting to access another user's vault item returns 404 Not Found	PASS ✓
Blank Secret on Update	Submitting an edit form with a blank secret preserves existing ciphertext	PASS ✓
Blank Notes on Update	Leaving notes blank preserves them; empty string clears them to null	PASS ✓
API Missing Token	Calling /api/v1/vault without a token returns HTTP 401 JSON (no redirect)	PASS ✓
API ?decrypt=true	Supplying ?decrypt=true query parameter is rejected with HTTP 400	PASS ✓
Account Lockout	5 failed login attempts trigger a 15-minute account lockout (HTTP 423)	PASS ✓
HSTS Header	Production environment emits Strict-Transport-Security header	PASS ✓
Encryption at Rest	Database inspection confirms only ciphertext is stored in Secret/Notes columns	PASS ✓
Key Persistence	Application restart does not invalidate existing ciphertext (Key ring persists)	PASS ✓
Global Exception Handling	Unhandled exceptions return JSON 500 for API and custom error page for MVC	PASS ✓
Mobile Responsiveness	Bootstrap 5 layout adapts cleanly across mobile and desktop viewports	PASS ✓

11. References & Bibliography

- **[1] Microsoft Docs:** ASP.NET Core 10 Overview, Entity Framework Core 10, and ASP.NET Core Identity documentation. https://learn.microsoft.com/
- **[2] ASP.NET Core Security:** Data Protection API Architecture and Key Management. https://learn.microsoft.com/en-us/aspnet/core/security/data-protection/
- **[3] OWASP Foundation:** Top Ten Web Application Security Risks — A01: Broken Access Control (IDOR) & Authentication Best Practices. https://owasp.org/
- **[4] Web Cryptography API:** W3C Recommendation for crypto.getRandomValues(). https://www.w3.org/TR/WebCryptoAPI/
- **[5] Course Curriculum:** Full Stack Web Development Project using ASP.NET Core (DSC-M-BCA-471P), guided by Prof. Sejal Shah at Lokmanya College of Computer Applications (Gujarat University), Academic Year 2026–27.